Revenue & Usage Page: Complete Data Guide and Accuracy Check

Admin dashboard, Revenue & Usage (Usage Products and Subscription Products tabs). Second, independent review carried out on 30 October 2026 against the live code on the main branch (commit e93b48df).

1. Verdict in one paragraph

The page's numbers are calculated correctly from the data the platform stores. I traced every figure back to the database row it comes from and re-checked every place in the system that writes those rows. No calculation error, double count, or missing source was found. The numbers can still *look* wrong if they are read with the wrong expectation, and section 6 lists each case and explains why the figure is right. One honest limit remains: the Cost column is the price charged to clients, not what JewlX pays its AI suppliers (section 7).

What was checked this time

Check	Result
Every AI feature is billed through the same helper and uses one of six known feature names	Yes. ring_chat, help_chat, ring_preview, grillz_preview, enquiry_preview, cost_estimate. No AI function writes usage any other way.
Every way money enters a wallet is classified	Yes. Stripe credit pack (topup), auto-recharge, admin top-up (credit). No other positive row type exists in the code. The old welcome-bonus trigger that put $2.50 into purchased wallets was removed in April 2026.
Every way money leaves a wallet is classified	Yes. AI usage (6 features), 365-day credit expiry, Stripe refund claw-back, admin reset-to-zero, admin pause. Only the 6 AI features count as Cost.
One wallet row and one subscription row per client	Yes, both have a unique index on client_id, so a client's figures can never be duplicated by a join.
Archived usage counts cannot be double counted	Yes. The archive job adds old rows to the monthly summary and then deletes them from the live log in the same step.
Internal/test flag matches the admin dashboard	Yes, both use the same clients.exclude_from_admin_revenue flag.
Page code and database function still match	Yes. Nothing in the page was changed since 21 Oct. The generated type file, which Lovable refreshes from the live database (last on 29 Oct), still carries the new function shape, which indicates the migration is applied.

2. How the AI credit money flows (the model behind the page)

Every sub-account has an **AI wallet**. Two kinds of credit live beside each other:

- **Purchased credits** (real money). A client buys a credit pack on Stripe, or an auto-recharge fires when the balance is low. An admin can also add credits by hand. Purchased credits expire 365 days after the last top-up.
- **Free monthly credits** (not money). Every account gets a monthly allowance (currently $2.50 unless changed in AI Cost settings). These sit in a separate table and are spent first.

Each AI request (a chat reply, an image render, a cost estimate) is charged at the **client price** = base cost x multiplier from the AI Cost settings. The charge is taken from free credits first, then from purchased credits. Every charge writes a transaction row, and every request also writes a usage-log row. The page is built from those two row types.

Event	What is written	How the page treats it
Stripe credit pack bought	Positive wallet row, type topup	Revenue (paid)
Auto-recharge fires	Positive wallet row, type auto_recharge	Revenue (paid)
Admin adds credits by hand	Positive wallet row, type credit	Revenue, shown separately as admin-granted; excluded when Paid top-ups only is ON
Stripe refund of a credit purchase	Negative wallet row, Refund clawback	Subtracted from Revenue
AI chat reply	Negative wallet row + usage log, feature ring_chat / help_chat	Cost (chat) and Chats count
AI image render	Negative wallet row + usage log, feature *_preview	Cost (image) and Images count
Cost estimate	Negative wallet row + usage log, feature cost_estimate	Cost (estimate) and Estimates count
Charge paid with free credits	Same negative row, description ends in (free credits)	Still Cost; also shown as paid with free credits
Purchased credits expire after 365 days	Negative wallet row, Purchased credits expired	Non-usage debit, NOT Cost
Admin resets or pauses a wallet	Negative wallet row, Admin: ...	Non-usage debit, NOT Cost
Free monthly grant arrives	Row in the free-credit table only	Not Revenue, not shown as a transaction

3. Usage Products tab, item by item

Everything on this tab is limited to the selected date range, except the Wallet column and the free-credit figures in the drawer, which are live balances.

Filters and switches

Control	What it does	Default
Sub-account	Shows one client, or all. Totals above the table follow the selection.	All Sub-accounts
Product (All / AI Chat / AI Image / Cost Estimate)	With one product chosen, the list keeps only clients who used that product, and the Cost tile and column show that product's cost only. Profit and Margin show a dash, because a top-up is not tied to a product.	All Products
Start date / End date	Both days are included in full (00:00 to 23:59 local time).	Last 120 days to today
Paid top-ups only	ON: Revenue = Stripe credit packs + auto-recharges only. OFF: admin-granted credits are added too. Free monthly credits are never revenue in either position.	OFF
Include internal / test accounts	ON adds JewlX's own and other flagged test accounts to the list and totals. They carry an internal badge.	OFF
Refresh	Re-runs the query with the current range, so rows created since the page opened appear.	
Export	Downloads the visible rows as CSV with every underlying field, including the breakdown fields not shown in the table.	

The four KPI tiles

Tile	Formula	Source rows
Total Revenue	Paid top-ups + admin-granted credits (unless Paid top-ups only) - refund claw-backs	wallet_transactions, positive rows and refund rows, created inside the range
Total Cost	Chat cost + Image cost + Estimate cost, at the client price	wallet_transactions, negative rows whose feature is one of the six AI features, created inside the range
Total Profit	Total Revenue - Total Cost	Calculated
Profit Margin	Total Profit / Total Revenue x 100. Shows a dash when Revenue is 0.	Calculated

Directly under the tiles a grey sentence prints the formula with the live amounts: paid top-ups, admin-granted credits, refunds, how much of the cost was paid with free credits, and any non-usage debits that were left out of Cost. If the manager wants to audit a total, this line is the place to start.

Table columns

Column	Meaning	Source
Sub-account	Client company name. An internal badge marks flagged test accounts.	clients.company_name
Plan	Subscription plan name, or a dash if none.	subscription_plans via client_subscriptions
Revenue	Money in for this client in the range, same formula as the tile. Bar is relative to the largest client.	wallet_transactions
Cost	AI usage charged in the range (or the chosen product's share).	wallet_transactions

Column	Meaning	Source
Chats	Number of AI chat replies (ring chat + help chat) in the range.	ai_usage_logs + ai_usage_monthly_summary
Images	Number of AI image renders (ring, grillz, enquiry) in the range.	same
Estimates	Number of cost estimates in the range.	same
Wallet	Purchased credit balance right now. Free credits are not included here (see drawer).	client_wallets.balance (live)
Profit	Revenue - Cost for this client. Green if positive, red if negative.	Calculated
Margin	Profit / Revenue. Dash when the client had no revenue in the range.	Calculated

Client drawer (click any row)

Shows plan, purchased balance, free credits left, the date purchased credits will expire, the range Revenue / Cost / Profit / Margin, then a **Where the numbers come from** box: paid top-ups, admin-granted credits, refund claw-backs, chat / image / estimate cost with their counts, the free-credit share, and non-usage debits. Below that, the last 50 wallet transactions of all time with their type. Adding up the transaction amounts inside the date range reproduces the client's Revenue and Cost.

4. Subscription Products tab, item by item

This tab is a snapshot of today, so the date range does not apply and the date pickers are hidden. It reads each client's subscription record and plan price.

Item	Meaning	Source / formula
Active Subscriptions (tile)	Number of clients whose subscription status is active.	client_subscriptions.status = active
Monthly Recurring Revenue (tile)	Sum of the monthly plan price of all active subscriptions.	subscription_plans.price_monthly, status active only
Est. Lifetime Revenue (tile)	Estimated total plan fees billed so far across all clients.	Sum of the column below
ARPU (tile)	Average monthly revenue per active subscription.	MRR / Active Subscriptions
Plan / Price	Plan name and its monthly list price.	subscription_plans
Status	active, trialing, past_due, canceled, none. 'cancels' is added when cancellation is scheduled for period end.	client_subscriptions.status, cancel_at_period_end
MRR (column)	Plan price if active, otherwise a dash.	as tile
Est. Lifetime Rev. (column)	Plan price x number of 30-day periods from the end of the trial (or the start, if no trial) until today, or until the cancellation date. Counted only for active, past_due and canceled rows. Trial-only and 'none' rows show a dash.	client_subscriptions.created_at, trial_end, updated_at, status
Started	When the subscription record was created.	client_subscriptions.created_at
Billing since	The date billing is assumed to have started (trial end or start). Blank when no period was billed.	Derived
Renews	End of the current billing period.	client_subscriptions.current_period_end

Important: Est. Lifetime Revenue is an estimate from plan price and elapsed time. The platform does not store Stripe invoice totals for plan fees, so actual collected subscription money must be read in the Stripe dashboard.

5. Accuracy checks performed

Live database access from the development PC is not available (the Supabase tooling on this machine is signed into a different project, and the browser extension is disconnected). The check below was therefore done by reading the current code of every component in the chain, and the results are stated as code facts. Section 8 gives a query the team can run on the live database to confirm the totals.

#	Question	Finding
1	Does every AI request create exactly one usage-log row and one (or a split pair of) wallet charge rows?	Yes. One shared helper does both, in that order, for all six functions (ring-chat, jewelx-help, generate-ring-preview, generate-grillz-preview, generate-enquiry-preview, estimate-lead-cost).
2	Can the feature name in a charge row fail to match the classification?	No. The description is always 'feature (model)' and the metadata carries the feature. The page reads the metadata first and the description as a fallback.
3	Can a client's figures be doubled by the joins?	No. client_wallets and client_subscriptions each have a unique client_id index.
4	Can archived months be counted twice?	No. The archive job inserts into the monthly summary and deletes the live rows in the same transaction.

#	Question	Finding
5	Is any positive wallet row left unclassified?	No. Only topup, auto_recharge and credit are written anywhere in the code; the first two are paid, the third is admin-granted.
6	Is any negative wallet row wrongly counted as Cost?	No. Expiry, refund, admin reset and admin pause all fall into the non-usage bucket and are printed separately.
7	Are refunds handled once, not twice?	Yes. A refund row is subtracted from Revenue and is excluded from Cost.
8	Do the range edges behave?	Yes. Start and end days are sent as full days in the browser's local time.
9	Are the two tabs reading the same internal flag as the admin dashboard?	Yes.

6. Figures that can look wrong but are correct

These are the patterns most likely to raise a question. Each one is expected behaviour of the data, not a bug.

1. A client shows Cost but $0 Revenue, so a negative Profit

The client is running on free monthly credits (or on credits bought before the date range). Usage is charged, but no money came in during the range. The drawer line 'of which paid with free credits' shows how much of the cost was free credit. This is the normal picture for accounts on the free tier.

2. Platform Total Profit is negative

Same reason at platform level. If most accounts live on free credits, their usage is real cost with no matching revenue. Switch the range to cover the top-up dates to see paying clients' true margin, or read Revenue and Cost per client rather than the platform total.

3. Revenue appears in a month with no usage, or usage appears in a month with no revenue

Top-ups are lump sums; usage is spread over the following weeks or months. A 30-day window can catch one without the other. Use a longer range for margin questions.

4. Chats are 0 for a client who clearly chats

Check two things: (a) is the client's widget using the AI chat at all (Ring OS chat is optional per client), and (b) the range. Counts now include the archive, so counts older than 90 days are no longer lost.

5. Counts and Cost do not match exactly for one client

Two legitimate reasons. First, if a request succeeds but the wallet is empty at the moment of charging, the usage row is written and the charge is refused, so there is a count without a cost. Second, the monthly archive is counted whole: a range that starts or ends mid-month includes the full archived month's count.

6. A refund makes Revenue smaller than the Stripe payments you remember

Correct by design: the refunded amount is subtracted. The drawer shows the refund line separately.

7. Turning 'Paid top-ups only' ON lowers Revenue

The difference is exactly the admin-granted credits in the range (shown in the grey formula line). Those were never money.

8. A deleted or inactive client is still listed

History is kept so that past revenue is not lost from totals. A client with no transactions in the range simply shows zeros.

9. Wallet balance is lower than Revenue - Cost

Wallet is today's purchased balance, not a range figure. Credits spent before the range, expired credits and admin resets all reduce it without appearing in the range's Cost.

10. Est. Lifetime Revenue differs from Stripe

It is plan price x elapsed 30-day periods, not invoices. Discounts, failed payments, plan changes and proration are not known to the platform.

7. Known limits of the data

- **Cost is the client price.** Each charge is base cost x multiplier from AI Cost settings, so Cost is what clients were billed, not JewlX's supplier bill. A rough supplier figure is Cost divided by the multiplier in force at the time (the code's fallback assumes 1.8x). The exact supplier cost per request is not stored, so true supplier margin cannot be computed for past dates.
- **Subscription money is estimated.** Plan fees are not recorded as invoices in the platform; see section 4.
- **Three different 'revenue' numbers exist in the admin.** This page's Usage tab = AI credit money. This page's Subscription tab = plan fee estimate. The main admin dashboard revenue tile = Shopify / Ring OS order revenue from the orders table. They measure different things and should never be compared to each other.
- **Archive granularity.** Usage older than 90 days is kept per calendar month, so a range edge inside a month counts that month in full.

8. How to verify the page yourself

Quick check inside the admin (2 minutes)

- Open Revenue & Usage, pick one paying client in the Sub-account filter and set a short range (for example the month of their last top-up).
- Click the row. In the drawer, add up the transactions dated inside the range: positive rows should equal Revenue, the AI-feature negative rows should equal Cost, and any expiry / admin rows should appear in the non-usage line, not in Cost.
- Toggle Paid top-ups only. Revenue should drop by exactly the admin-granted amount printed in the grey formula line.
- Compare the Chats / Images counts with the same client's AI Usage screen for the same dates.

Database cross-check (for the developer, Supabase SQL editor)

Run with the same start and end as the page. The two result sets should match the page's Total Revenue and Total Cost with 'Paid top-ups only' OFF and internal accounts OFF.

```
-- Replace the two dates with the page range (inclusive days, UTC).
WITH r AS (SELECT '2026-07-01'::timestamptz AS p_start, '2026-10-30 23:59:59'::timestamptz AS p_end)
SELECT
  SUM(wt.amount) FILTER (WHERE wt.amount > 0 AND wt.type IN ('topup','auto_recharge'))        AS paid,
  SUM(wt.amount) FILTER (WHERE wt.amount > 0 AND wt.type NOT IN ('topup','auto_recharge'))    AS admin_granted,
  SUM(ABS(wt.amount)) FILTER (WHERE wt.description ILIKE 'Refund clawback%')                 AS refunds,
  SUM(ABS(wt.amount)) FILTER (WHERE wt.type = 'debit'
       AND COALESCE(wt.metadata->>'feature', split_part(wt.description,' ',1))
           IN ('ring_chat','help_chat','ring_preview','grillz_preview',
               'enquiry_preview','cost_estimate'))                                           AS ai_cost,
  SUM(ABS(wt.amount)) FILTER (WHERE wt.amount < 0
       AND wt.description NOT ILIKE 'Refund clawback%'
       AND COALESCE(wt.metadata->>'feature', split_part(wt.description,' ',1))
           NOT IN ('ring_chat','help_chat','ring_preview','grillz_preview',
                   'enquiry_preview','cost_estimate'))                                       AS non_usage_debits
FROM wallet_transactions wt
JOIN clients c ON c.id = wt.client_id, r
WHERE wt.created_at BETWEEN r.p_start AND r.p_end
  AND NOT COALESCE(c.exclude_from_admin_revenue, false);
-- Page: Total Revenue = paid + admin_granted - refunds ; Total Cost = ai_cost

-- Usage counts (live log + archive), same range:
WITH r AS (SELECT '2026-07-01'::timestamptz AS p_start, '2026-10-30 23:59:59'::timestamptz AS p_end)
```

```
SELECT feature, SUM(n) FROM (
  SELECT l.feature, COUNT(*) AS n FROM ai_usage_logs l, r
   WHERE l.created_at BETWEEN r.p_start AND r.p_end GROUP BY l.feature
  UNION ALL
  SELECT s.feature, s.total_requests FROM ai_usage_monthly_summary s, r
   WHERE s.month >= date_trunc('month', r.p_start)::date AND s.month <= r.p_end::date
) x GROUP BY feature ORDER BY feature;
```

9. Deployment state

- Fix commit 9c06c41e (21 Oct 2026) is on main and auto-deployed. No file of this page has changed since.
- The database migration 20261021120000 appears to be applied: Lovable's regenerated type file from the live database (29 Oct) still shows the new function shape. Confirmation: if the page loads without an error banner, the migration is live, because the page now sends a parameter the old function did not accept.
- Out of scope but worth a ticket: the monthly free-credit cron function still references two renamed columns and likely fails each month; clients still receive their free grant through a fallback at first use. It does not affect any figure on this page.